Wire-Dependency DAGs for Size-Generalizing Quantum Compiler Models

Anonymous Authors
Affiliation withheld for review

November 2, 2026

Abstract

Learned quantum-compiler components are often trained on smaller circuits than those encountered at deployment, making the circuit encoding a scaling decision. We compare a wire-dependency DAG network with a parameter-matched canonical layer-grid GRU on exact depth-critical gate prediction. Both receive identical gate and operand features. On random circuits, same-size evaluation favors the grid (0.9406 versus 0.8556 AUROC), yet DAG wins all three paired seeds after scaling from 4–8 to 9–12 qubits (0.8078 versus 0.7861). A fresh five-seed study on QAOA-like, VQE, and arithmetic circuits replicates the result: simultaneous width-and-depth shift yields 0.8902 AUROC and 0.8351 F1 for DAG versus 0.8495 and 0.7928 for grid. An axis decomposition attributes most of the gain to added qubit wires (+0.0514 AUROC); depth-only gain is +0.0025. Both encoders transfer poorly to an unseen chemistry-inspired family, where grid AUROC is higher. Across two frozen studies and eight paired seeds, explicit wire dependencies improve within-family size transfer, especially under width growth. The result supports dependency-aware encoders for learned compiler components but does not claim a new critical-path algorithm or production optimization advantage.

1 The contribution

A quantum circuit has a visual layout, an operation sequence, and a dependency graph. These encodings contain related information, but they make different regularities easy for a neural model to learn. A layer grid exposes stable coordinates such as "wire 2, layer 5." A wire-dependency DAG exposes relations such as "this CX follows the previous operation on its control wire." Grid coordinates change as width and depth grow; the meaning of a predecessor edge does not.

Size-transfer hypothesis. After training on 4–8-qubit circuits, the wire DAG will achieve higher AUROC than a matched layer grid on 9–12-qubit circuits.

The ranking reverses: the grid leads by 0.0850 AUROC at the training sizes, while the DAG leads by 0.0217 AUROC on larger circuits and wins every seed.

Concretely, this paper contributes a controlled model-selection result. Same-size evaluation would select the grid because its AUROC is 0.9406 versus 0.8556 for the DAG. Larger-circuit evaluation reverses that choice: the DAG reaches 0.8078 versus 0.7861 for the grid. The data, node features, parameter counts, training updates, and paired seeds are matched, so the comparison isolates the effect of circuit representation under the tested size shift.

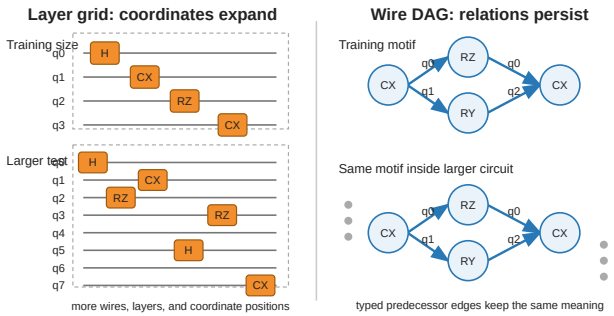


Figure 1: Schematic motivation. When a circuit gains wires and independent operations, its grid becomes wider and longer and a gate occupies a different coordinate. The corresponding DAG still connects that gate to its immediate predecessors on the same wires. The diagram motivates the size-shift test; it is not itself an experimental result.

The depth-critical label is exactly computable and is used to remove label and simulator noise from this comparison. We are not proposing a learned critical-path algorithm. We use critical-path membership as a stress test whose answer depends on the wire relations encoded by the DAG.

The first study uses random circuits and reveals the in-distribution ranking reversal. The second is a fresh replication on three structured circuit families with five paired seeds. It changes width and depth separately, then together, and reserves chemistry-inspired circuits as an unseen family. The replication both strengthens and narrows the conclusion: DAG improves transfer when known circuit families become wider, while a new family remains a separate generalization problem.

2 Why size transfer matters

Quantum machine-learning systems frequently train on smaller instances than those of ultimate interest. Clas-

sical simulation cost grows rapidly with qubit count, hardware data are limited, and compiler searches can be expensive. Frameworks for quantum machine learning make circuit data accessible [1], but a model that interpolates among familiar sizes may still rely on coordinates that do not persist at deployment scale.

The issue is especially relevant to compilation. Circuit depth, two-qubit gates, and routing overhead constrain near-term workloads [2, 3, 4]. Many learned compiler components consume a circuit encoding before predicting cost, selecting transformations, or guiding search. Their first design choice is therefore representational: flatten the circuit, place it on a grid, or preserve dependencies as a graph.

Graph message passing supplies a relational inductive bias [5, 6]. It can also be harder to optimize and may discard useful global coordinates. We should not expect a graph to win everywhere. The scientifically useful question is where its bias helps and what an in-distribution benchmark would miss.

Our study contributes:

1. a matched DAG-versus-grid experiment that changes circuit size rather than merely resampling circuits of the same shape;
2. a frozen three-seed result showing that representation ranking reverses between interpolation and size extrapolation;
3. a mechanism check for legal topological reordering and strong sequence, Transformer, canonical-grid, and recurrent-DAG development controls;
4. a complete record of protocols, split hashes, negative trials, and follow-up tasks that did not establish a practical optimizer advantage.

3 Controlled representation probe

3.1 Wire-dependency labels

Let a circuit contain operations $V = \{v_i\}_{i=1}^{m}$ on n logical qubits. For every qubit wire, connect consecutive operations that touch that wire. Their union is a directed acyclic graph $G = (V, E)$. Each edge retains its wire identity. Independent operations have no edge between them, even if one happens to be serialized first.

Let $f(v)$ be the longest predecessor path ending at v, $b(v)$ the longest successor path beginning at v, and $D = \max_v f(v)$. The exact node label is

$$y_v = \mathbf{1}[f(v) + b(v) - 1 = D]\,. \tag{1}$$

Thus, $y_v = 1$ when gate v belongs to at least one maximum dependency path. The label is a scheduling property of the complete circuit, not a local gate attribute.

Consider two chains that merge at a CX: $H(q_0) \to CX(q_0, q_1)$ and $R_Z(q_2) \to R_Y(q_2)$, followed by $CX(q_1, q_2)$. Operations on the two chains can be interleaved in several legal sequences. Their indices and grid occupancy vary, while their typed predecessor relations and critical-path calculation remain unchanged. Larger circuits add more such independent and merging chains.

3.2 Matched encoders

Every operation receives the same features in both models: one-hot gate type, membership of each wire, CX control and target roles, sine and cosine of the rotation angle, and arity. The maximum feature width supports the full 12-qubit evaluation range.

The **DAG model** adapts LayerDAG-style sparse graph aggregation [7] to circuit dependencies. It projects features to width 64 and applies six residual bidirectional message-passing blocks. Forward messages come from predecessors, backward messages come from successors, and the wire identity is concatenated to every message. A linear head predicts one logit per gate.

The **grid model** computes deterministic earliest dependency layers, then orders gates within each layer by operands, type, and parameters. A two-layer bidirectional GRU of width 96 processes this canonical sequence and predicts the same node labels. Canonicalization makes this a strong control: it removes arbitrary serialization noise rather than giving the DAG an artificially weak sequence baseline.

The DAG has 87,745 trainable parameters and the grid 88,417, a 0.77% difference. Both receive 6,000 updates, the same batches, AdamW optimizer, node features, labels, and validation-only checkpoint selection.

4 Experimental firewall

Deterministic circuits contain H, RY, RZ, and directed CX operations. Training and validation use 4–8 qubits and 24–56 gates. The in-distribution test uses the same range with a fresh seed. The size-shifted test uses 9–12 qubits and 64–96 gates. Training, validation, same-size test, reordered test, and larger test have distinct stored SHA-256 hashes.

Development considered raw bidirectional sequence, token Transformer, canonical-grid GRU, fixed-depth DAG, and full-depth recurrent-DAG encoders. The final comparison froze the selected six-layer DAG and canonical grid, all data ranges, 6,000 updates, metrics, and success rule before final data generation. Three paired seeds (51, 52, and 53) train on identical examples.

The predeclared size-transfer claim required (i) positive DAG-minus-grid larger-circuit AUROC for every seed, (ii) mean improvement of at least 0.015 AUROC, and (iii) positive mean F1 improvement. Same-size superiority was not required. We report node-level AUROC and F1, with mean and sample standard deviation across seeds.

The structured replication uses a separate development/final firewall. Training contains 1,000 balanced

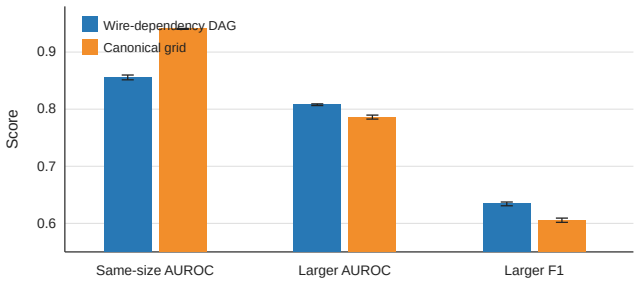


Figure 2: The central result. In-distribution evaluation selects the grid; larger-circuit evaluation selects the DAG. Error bars are sample standard deviations over three paired seeds.

Table 1: Frozen final results (mean ± sample standard deviation).

Encoding	Same AUROC	Larger AUROC	Larger F1
Wire DAG	0.8556 ± .0042	**0.8078 ± .0015**	**0.6341 ± .0033**
Layer grid	**0.9406 ± .0008**	0.7861 ± .0035	0.6055 ± .0037

QAOA-like [8], hardware-efficient VQE [9], and reversible linear-arithmetic circuits with 4–7 qubits and 32–64 gates. Five fresh paired seeds (81–85) receive 4,000 updates. Each 512-circuit final split isolates same size, width shift (8–12 qubits), depth shift (72–104 gates), simultaneous width and depth shift, or an unseen chemistry-inspired family. The final gate requires DAG to win simultaneous-shift AUROC in all five seeds with mean gain at least 0.015, improve simultaneous-shift F1, and improve width-shift AUROC.

5 Results across two circuit distributions

Table 1 shows the reversal. The grid is substantially better on the same size distribution. This is evidence that its canonical coordinates are useful, not that it is a weak baseline. Under size shift, however, paired DAG-minus-grid AUROC differences are +0.0202, +0.0177, and +0.0274. Their mean is +0.0217, exceeding the frozen +0.015 gate. Larger-circuit F1 improves by +0.0286 on average. Every seed favors the DAG on both shifted metrics.

Descriptive seed-level 95% t intervals are [0.0093, 0.0342] for the AUROC difference and [0.0219, 0.0353] for the F1 difference. With three seeds, these intervals describe consistency rather than supplying strong asymptotic inference. The all-seed result and frozen effect-size criterion are the primary evidence.

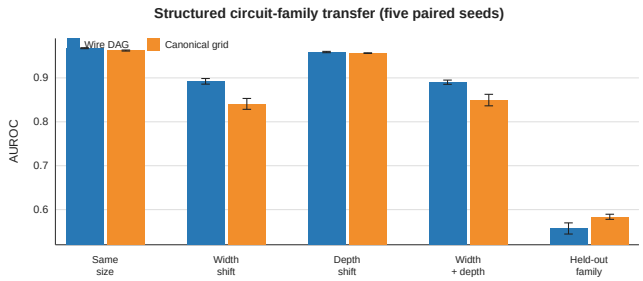


Figure 3: Fresh five-seed structured-circuit results. DAG's advantage is concentrated in splits that add qubit wires. Error bars show sample standard deviation across paired seeds.

Table 2: Structured final AUROC (five-seed mean ± sample standard deviation).

Split	Wire DAG	Layer grid	Difference
Same size	0.9677 ± .0011	0.9623 ± .0012	+0.0054
Width shift	**0.8922 ± .0064**	0.8408 ± .0124	**+0.0514**
Depth shift	**0.9589 ± .0015**	0.9564 ± .0007	+0.0025
Width + depth	**0.8902 ± .0048**	0.8495 ± .0131	**+0.0407**
Held-out chemistry	0.5571 ± .0127	**0.5837 ± .0058**	−0.0266

5.1 Structured-family replication

The structured confirmatory gate passes. Simultaneous-shift paired AUROC differences are +0.0321, +0.0243, +0.0533, +0.0414, and +0.0525. DAG also improves simultaneous-shift F1 from 0.7928 to 0.8351. The width-only gain is +0.0514 AUROC, whereas the depth-only gain is +0.0025. Thus, adding qubit wires is the dominant tested source of representation advantage. The unseen-family result reverses: grid leads chemistry AUROC by 0.0266 and both AUROCs are below 0.59. Size transfer within known families does not imply family transfer.

5.2 What explains the result?

The grid can learn that certain canonical positions are commonly critical within the training size range. Wider circuits change wire-membership patterns, the number of simultaneously active chains, and merge locations. The five-seed decomposition shows that this width change matters much more than merely extending circuit depth. The bidirectional GRU can process the larger canonical sequence, but it must extrapolate positional patterns across new wire configurations.

The DAG performs the same six local dependency updates regardless of total serialization length. A predecessor message still means "the previous operation on this wire" on a 12-qubit circuit. This does not make the model an exact longest-path solver; six layers cannot propagate across an arbitrarily long graph. It does make its learned features depend on relations whose semantics remain sta-

ble when the circuit grows.

A legal topological-reordering test supports this mechanism: DAG logits agree within 10^{-6} across reorderings. The canonical grid is also stable because it explicitly recanonicalizes. A raw sequence control is not stable and reached only 0.7303 AUROC on larger development circuits, compared with 0.7687 for the canonical grid and 0.7933 for the selected DAG at that development stage. A token Transformer and recurrent DAG were weaker development controls. These development comparisons motivated the final architectures; only Table 1 is confirmatory.

6 What the result does and does not mean

The confirmed statement is:

> Across random circuits and structured QAOA-like, VQE, and arithmetic families, explicit wire-dependency message passing transfers better than a parameter- and update-matched canonical layer grid when test circuits contain more qubit wires. This result holds for all eight paired seeds across the two frozen studies.

This establishes neither universal DAG superiority nor a new quantum compiler. Critical paths are exactly computable in linear time, so using a neural network for this label alone would be unnecessary. The value of the task is experimental control: it reveals whether an encoder learns dependency structure that survives a size change.

We attempted three downstream checks. Fidelity-safe single-gate deletion was saturated by a smallest-angle heuristic. Controlled identity-rewrite prioritization favored DAG in all three seeds (51.94% versus 48.15% oracle depth recovered) but missed its predeclared five-point effect-size gate. A structured routed-rewrite development task used verified safe templates from QAOA-like, VQE, arithmetic, and chemistry-inspired circuits; its candidate-aware grid remained stronger (6.664 versus 6.305 mean routed-CX reduction), so no final routed test was opened. These failures prevent a practical optimizer claim. They do not alter the frozen representation result, which asks a different question.

The next practical test should predict a cost that is expensive to compute directly from the source DAG, such as post-routing performance over many transformations, calibrated noise-aware utility, or synthesis interactions. It should use public circuit families, several hardware graphs, and enough repeated workloads to amortize training cost.

7 Limitations and reproducibility

Circuits are deterministic and synthetic, although the replication uses QAOA-like, VQE, arithmetic, and chemistry-inspired structures. They contain at most 12 qubits. The task probes one topological property. Parameter counts and updates are matched, but neither model receives an exhaustive independent hyperparameter search. The native PyTorch DAG is source-informed by LayerDAG-style message passing; DGL runtime parity is not claimed. The failed chemistry transfer shows that structural family shift remains unsolved.

Reproducibility artifacts include the pre-final protocol, deterministic generators, split hashes, training histories, parameter counts, every development and final metric, failed downstream studies, and a compute ledger. Twenty-nine tests cover gate semantics, canonical scheduling, graph aggregation, topological equivariance, fidelity labels, and exact rewrite templates. The complete internally timed CPU training lower bound exceeds 190 minutes; no GPU or paid resource was used.

8 Conclusion

Across random and structured circuits, eight paired seeds support the same central conclusion: explicit wire dependencies improve critical-gate prediction when known circuit distributions grow in size. The structured decomposition identifies circuit width as the main driver. In-distribution ranking is distribution dependent, and unseen-family chemistry transfer remains poor. Explicit dependency structure is therefore a justified inductive bias for within-family size extrapolation, not a universal circuit representation or a demonstrated production optimizer.

References

[1] Michael Broughton, Guillaume Verdon, Trevor McCourt, Antonio J. Martinez, Jae Hyeon Yoo, Sergei V. Isakov, Philip Massey, Ramin Halavati, Murphy Yuezhen Niu, Alexander Zlokapa, Evan Peters, Owen Lockwood, Andrea Skolik, Sofiene Jerbi, Vedran Dunjko, Martin Leib, Michael Streif, David Von Dollen, Hongxiang Chen, Shuxiang Cao, Roeland Wiersema, Hsin-Yuan Huang, Jarrod R. McClean, Ryan Babbush, Sergio Boixo, Dave Bacon, Alan K. Ho, Hartmut Neven, and Masoud Mohseni. Tensorflow quantum: A software framework for quantum machine learning. *arXiv preprint arXiv:2003.02989*, 2021.

[2] John Preskill. Quantum computing in the NISQ era and beyond. *Quantum*, 2:79, 2018.

[3] Gushu Li, Yufei Ding, and Yuan Xie. Tackling the qubit mapping problem for NISQ-era quantum devices. *ASPLOS*, pages 1001–1014, 2019.

[4] Alexander Cowtan, Silas Dilkes, Ross Duncan, Will Simmons, and Seyon Sivarajah. On the qubit routing problem. *Proceedings of the 14th Conference on*

the Theory of Quantum Computation, Communication and Cryptography, 2019.

[5] Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. Neural message passing for quantum chemistry. In *Proceedings of the 34th International Conference on Machine Learning*, pages 1263–1272, 2017.

[6] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations*, 2017.

[7] Mufei Li, Viraj Shitole, Eli Chien, Changhai Man, Zhaodong Wang, Srinivas Sridharan, Ying Zhang, Tushar Krishna, and Pan Li. LayerDAG: A layerwise autoregressive diffusion model for directed acyclic graph generation. In *International Conference on Learning Representations*, 2025.

[8] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. A quantum approximate optimization algorithm. *arXiv preprint arXiv:1411.4028*, 2014.

[9] Alberto Peruzzo, Jarrod McClean, Peter Shadbolt, Man-Hong Yung, Xiao-Qi Zhou, Peter J. Love, Alán Aspuru-Guzik, and Jeremy L. O'Brien. A variational eigenvalue solver on a photonic quantum processor. *Nature Communications*, 5:4213, 2014.